Recursive Time Complexity Examples

1.
```
public void test(int n) {
    if (n > 0) {
        System.out.printf("%d", n);
        test(n - 1);
    }
}
```

2.
```
public void test(int n) {
    if (n > 0) {
        for (int i = 0; i < n; i++) {
            System.out.printf("%d", n);
        }
        test(n - 1);
    }
}
```

3.
```
public void test(int n) {
    if (n > 0) {
        for (int i = 1; i < n; i = i * 2) {
            System.out.printf("%d", i);
        }
        test(n - 1);
    }
}
```

4.
```
public void test(int n) {
    if (n > 0) {
        System.out.printf("%d", n);
        test(n - 1);
        test(n - 1);
    }
}
```

5.
```
public void test(int n) {
    if (n > 1) {
        System.out.printf("%d", n);
        test(n / 2);
    }
}
```

6.
```
public void test(int n) {
    if (n > 1) {
        for (int i = 0; i < n; i++) {
            System.out.printf("%d", i);
        }
        test(n / 2);
    }
}
```

7.
```
public void test(int n) {
    if (n > 1) {
        for (int i = 0; i < n; i++) {
            System.out.printf("%d", i);
        }
        test(n / 2);
        test(n / 2);
    }
}
```

Recursive Time Complexity Solutions

1. **Recurrence Relation:** $T(n) = T(n-1) + O(1)$
   Time Complexity: $O(n)$

2. **Recurrence Relation:** $T(n) = T(n-1) + O(n)$
   Time Complexity: $O(n^2)$

3. **Recurrence Relation:** $T(n) = T(n-1) + O(\log n)$
   Time Complexity: $O(n \log n)$

4. **Recurrence Relation:** $T(n) = 2T(n-1) + O(1)$
   Time Complexity: $O(2^n)$

5. **Recurrence Relation:** $T(n) = T(n/2) + O(1)$
   Time Complexity: $O(\log n)$

6. **Recurrence Relation:** $T(n) = T(n/2) + O(n)$
   Time Complexity: $O(n)$

7. **Recurrence Relation:** $T(n) = 2T(n/2) + O(n)$
   Time Complexity: $O(n \log n)$